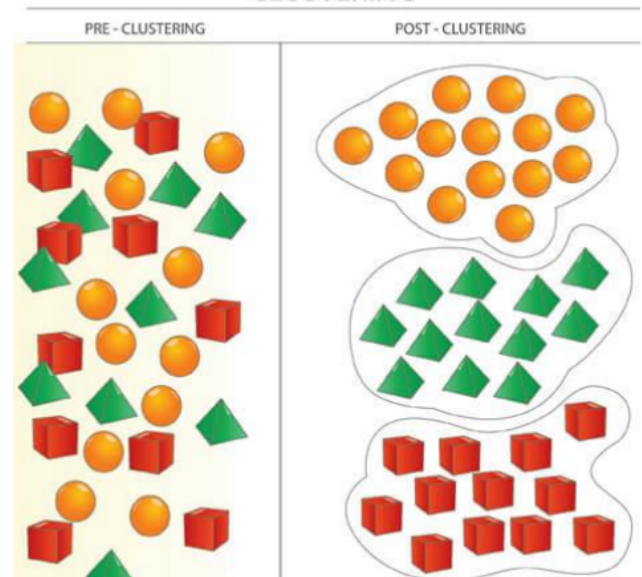


<https://drive.google.com/file/d/1DNcKodfdWU5Nh8ibPHYcjllstsJYaPx3/view>

## ★ Clustering:

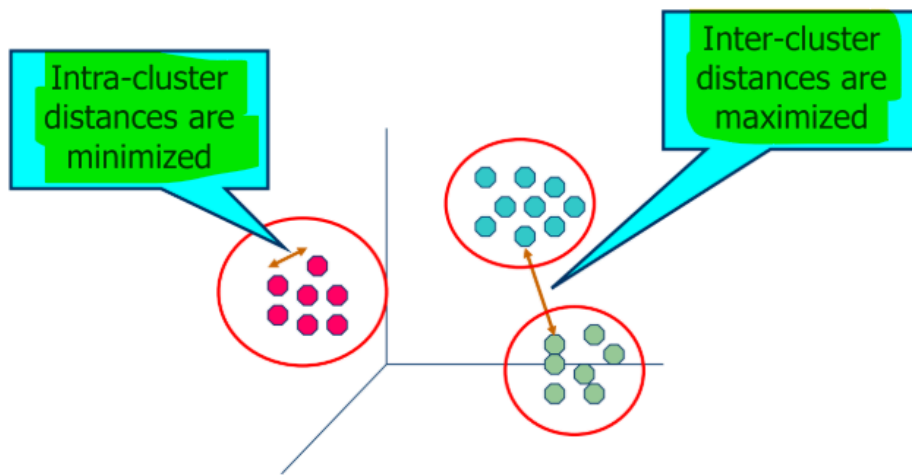
- **Unsupervised** learning
- Requires data, but no labels
- **Detect patterns**
- e.g. in
  - Group emails or search results
  - Customer shopping patterns
  - Regions of images
- Useful when don't know what you're looking for



# basic idea of clustering: **group similar instances**

# what could "similar" mean?

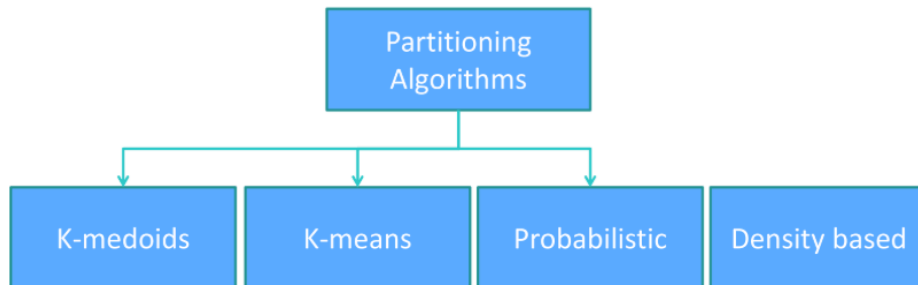
- Clustering depends heavily on **how you measure "closeness"**
- If you use straight-line distance (Euclidean), you might get one grouping
- If you use another measure, you might get completely different clusters



# ★ Clustering algorithms

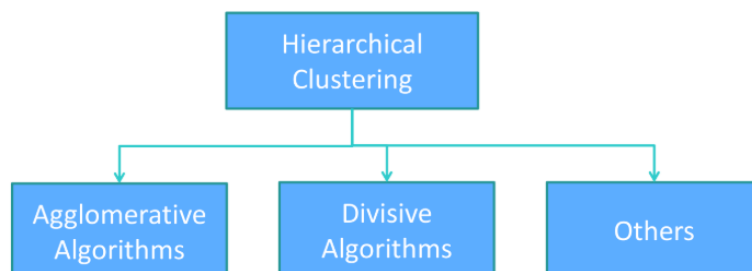
## 1. Partition / Flat Algos:

- 1) **K - means** clustering
- 2) Mixture of **Gaussian**
- 3) **Spectral** clustering



## 2. Hierarchical algos:

- 1) **Agglomerative** ⇒ **Bottom UP**
- 2) **Divisive** ⇒ **TOP DOWN**



# ★ Clustering examples

- **Image segmentation**
    - Goal: Break up the image into meaningful or perceptually similar regions
  - **Clustering gene expression data**
-

## ◆ Exclusive vs Non-Exclusive Clustering

### ✓ Exclusive Clustering

- Each data point belongs to **only one cluster**.
- **No overlap** between clusters.

### ✓ Non-Exclusive Clustering

- A data point can belong to **multiple clusters** at the same time.
  - Useful for:
    - **Multi-class membership**
    - Border points lying between clusters
- 

## ◆ Fuzzy vs Non-Fuzzy Clustering

### ✓ Non-Fuzzy (Hard) Clustering

- A point either belongs to a cluster or it doesn't.
- **Membership values** are only **0 or 1**.

### ✓ Fuzzy Clustering

- Every point belongs to **all clusters with different weights**.
  - Membership values range from **0 to 1**.
  - **Sum of all membership weights = 1**.
- 

## ◆ Partial vs Complete Clustering

### ✓ Complete Clustering

- **Every data point** is **assigned to a cluster**.

### ✓ Partial Clustering

- **Only selected data points are clustered.**
  - Some points remain unclustered.
- 

## ◆ Heterogeneous vs Homogeneous Clustering

### ✓ Homogeneous Clusters

- Clusters have **similar**:
  - **Size**
  - **Shape**
  - **Density**

### ✓ Heterogeneous Clusters

- Clusters **differ significantly** in:
  - **Size**
  - **Shape**
  - **Density**

**One cluster may be large and sparse, while another is small and dense.**

---

## Types of Clusters

### 1. Well-Separated Clusters

A cluster is a group of points where:

- **Every point is closer to points inside its own cluster**
  - Than to any point outside the cluster
  - ☒ Clear boundaries
  - ☒ Easy to identify
  - ☒ Little overlap
- 

## 2. Contiguity-Based (Nearest Neighbor or Transitive) Clusters

- A cluster is formed when **points are connected through nearby neighbors.**
- A point belongs to a cluster if it is **close to one or more points already in that cluster.**
- ☒ Forms chains of connected points
- ☒ Neighbor relationships are important

### Example

Even if two end points are far apart, they belong to the **same cluster through intermediate points.**

---

## 3. Density-Based Clusters

- A cluster is a **high-density region** separated from other clusters **by low-density regions.**
  - ☒ Handles irregular shapes
  - ☒ Works with intertwined clusters
  - ☒ Detects noise and outliers
-

## ● 4. Conceptual (Property-Based) Clusters

- Clusters are formed based on a **shared property, concept, or characteristic**, rather than distance.
  -  Objects **share common features**
  -  **Focuses on meaning** rather than location
- 

## ● 5. Objective Function-Based Clusters

- Clusters are created by **optimizing** (**minimizing** or **maximizing**) an **objective function**.
- The algorithm searches for the clustering that **gives the best score** according to a mathematical criterion.

### How It Works

1. Divide data into clusters.
2. Calculate a **goodness score**.
3. **Compare** possible **clusterings**.
4. **Select the best one**.

 Checking all possible clusterings is **NP-Hard** (computationally expensive).

### Types of Objectives

#### **Global Objective**

- **Optimizes** the clustering of the **entire dataset**.
- Common in **Partitional Clustering** (e.g., K-Means).

#### **Local Objective**

- **Optimizes decisions at smaller levels**.
- Common in **Hierarchical Clustering**.

## Model-Based Variation

Instead of directly forming clusters:

1. Assume data follows a statistical model.
  2. Estimate model parameters from data.
  3. **Create clusters based on the model.**
- 

## K-Means

- **Iterative** clustering algorithm
- groups data into **K clusters**
- you choose K beforehand
- Each cluster has a **center (called centroid)**.

### › Why “iterative”?

- doesn't get the answer in one step.
- It **repeats steps again and again** until **clusters stop changing**.
- **Stops** when **no point changes its cluster**

### › Step-by-Step Process

#### Step 1: Initialize (start randomly)

- Pick **K random points** as initial cluster centers (**centroids**).
- 

#### Step 2: Assignment Step

- **For each data point:**

- **Check** which **centroid** is **closest**
- Assign the point to that cluster

👉 So now every point belongs to one cluster.

---

### Step 3: Update Step

- For each cluster:
  - Compute the **average (mean)** of all points in it
  - **Move** the **centroid** to this **new average position**

👉 That's why it's called "**K-Means**" (mean = average).

---

### Step 4: Repeat (Iterate)

- Go back to:
    - **Assign points again** (using new centroids)
    - **Update centroids** again
- 

### step 5: When does it stop?

- When **no point changes its cluster**
- (or changes are **extremely small**)

👉 Means the clusters are now **stable**.

---

- Diff random starts → different results
- Works better when clusters r clear and roughly circular!

## 😞 Properties of K-Means Algorithm



## 1. Guaranteed Convergence

- K-Means will **always stop** after a **finite number of steps**

## 2. Running Time per Iteration

Each iteration has **2 main steps**:

### Step 1: Assign points to nearest centroid

- For each of the **N data points**, you **check** distance to **K centroids**
- So total work =  **$N \times K$**

👉 Time complexity:  **$O(KN)$**

### Step 2: Update centroids (compute mean)

- You go through all **N points** once to compute new averages

👉 Time complexity:  **$O(N)$**

## > Overall Idea of Efficiency

- Each iteration cost =  **$O(KN) + O(N)$**
- Simplified →  **$O(KN)$**  (dominant term)

## > Simple intuition

- **Assigning clusters** is the **expensive part** (checking all centroids)
- Updating centroids is **faster** (just averaging)

# ★ EXAMPLE : K-Means for Image Segmentation

- Instead of clustering points, we are clustering **pixels of an image**
- Goal: **divide the image into regions** where **pixels look similar (color, intensity, etc.)**

👉 This is called **image segmentation**

- Each pixel = a data point (based on its color values like RGB)
- K-Means groups pixels into **K clusters**
- Each cluster = one region in the image

K = 2

- Image is divided into **2 regions only**
- Very rough → only major areas (like foreground vs background)

👉 Result: **loss of detail**

---

K = 3

- Now **3 regions**
- More structure appears (some parts separated better)

👉 Slightly better segmentation

---

K = 10

- Many clusters → **more detailed regions**
- Colors and objects are more clearly separated

👉 **Looks closer to original image**

---

## 4. Bottom Row (percentages)

- These percentages (4%, 8%, 17%) show **how much information/detail is preserved**
- **Higher K → more detail → higher percentage**

K=2



**Goal of Segmentation is to partition an image into regions each of which has reasonably homogenous visual appearance.**

Original



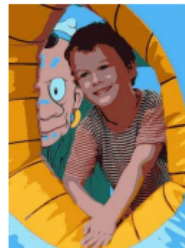
K=2



K=3



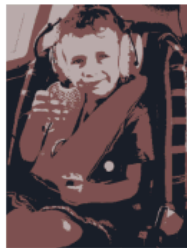
K=10



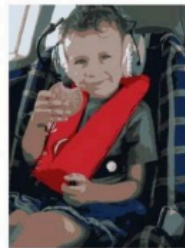
Original



4%



8%



17%



## ★ EXAMPLE 1:

### ◆ Given Data

$K = \{2, 3, 4, 10, 11, 12, 20, 25, 30\}$

Number of clusters:  $K = 2$

---

### ◆ Step 1: Initialize Centroids

We start with:

- $m_1 = 4$
- $m_2 = 12$

👉 These are our **starting centroids**

---

### ◆ Step 2: Assign Points (based on distance)

Check which centroid each number is closer to:

- Closer to **4**  $\rightarrow \{2, 3, 4\}$
- Closer to **12**  $\rightarrow \{10, 11, 12, 20, 25, 30\}$

👉 Clusters:

- $k_1 = \{2, 3, 4\}$
  - $k_2 = \{10, 11, 12, 20, 25, 30\}$
- 

### ◆ Step 3: Update Centroids (mean)

- $m_1 = (2+3+4)/3 = 3$
  - $m_2 = (10+11+12+20+25+30)/6 = 18$
- 

## ◆ Step 4: Reassign Points

Using new centroids (3 and 18):

- Closer to **3** → {2, 3, 4, 10}
- Closer to **18** → {11, 12, 20, 25, 30}

👉 Clusters:

- $k_1 = \{2, 3, 4, 10\}$
  - $k_2 = \{11, 12, 20, 25, 30\}$
- 

## ◆ Step 5: Update Again

- $m_1 = (2+3+4+10)/4 = 4.75$
  - $m_2 = (11+12+20+25+30)/5 = 19.6$
- 

## ◆ Step 6: Reassign Again

Using (4.75 and 19.6):

- Closer to **4.75** → {2, 3, 4, 10, 11, 12}
- Closer to **19.6** → {20, 25, 30}

👉 Clusters:

- $k_1 = \{2, 3, 4, 10, 11, 12\}$
- $k_2 = \{20, 25, 30\}$

---

### ◆ Step 7: Final Update

- $m_1 = (2+3+4+10+11+12)/6 = 7$
  - $m_2 = (20+25+30)/3 = 25$
- 

### ◆ Step 8: Check Again

Reassign using (7 and 25):

- $k_1$  stays  $\rightarrow \{2, 3, 4, 10, 11, 12\}$
- $k_2$  stays  $\rightarrow \{20, 25, 30\}$

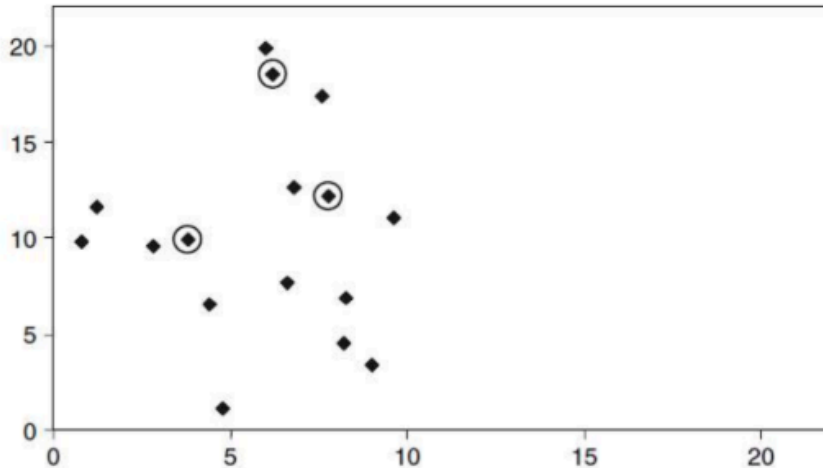
👉 No change  $\rightarrow$  STOP

---

### ✅ Final Answer

- **Cluster 1** =  $\{2, 3, 4, 10, 11, 12\}$
  - **Cluster 2** =  $\{20, 25, 30\}$
  - Final centroids:  $m_1 = 7, m_2 = 25$
-

- *k-means algorithm by using it to cluster the 16 objects with two attributes x and y*



| <i>x</i> | <i>y</i> |
|----------|----------|
| 6.8      | 12.6     |
| 0.8      | 9.8      |
| 1.2      | 11.6     |
| 2.8      | 9.6      |
| 3.8      | 9.9      |
| 4.4      | 6.5      |
| 4.8      | 1.1      |
| 6.0      | 19.9     |
| 6.2      | 18.5     |
| 7.6      | 17.4     |
| 7.8      | 12.2     |
| 6.6      | 7.7      |
| 8.2      | 4.5      |
| 8.4      | 6.9      |
| 9.0      | 3.4      |
| 9.6      | 11.1     |

•The columns headed *d1*, *d2* and *d3* in Figure shows the Euclidean distance of each of the 16 points from the three centroids.

•The column headed 'cluster' indicates the centroid closest to each point and thus the cluster to which it should be assigned.

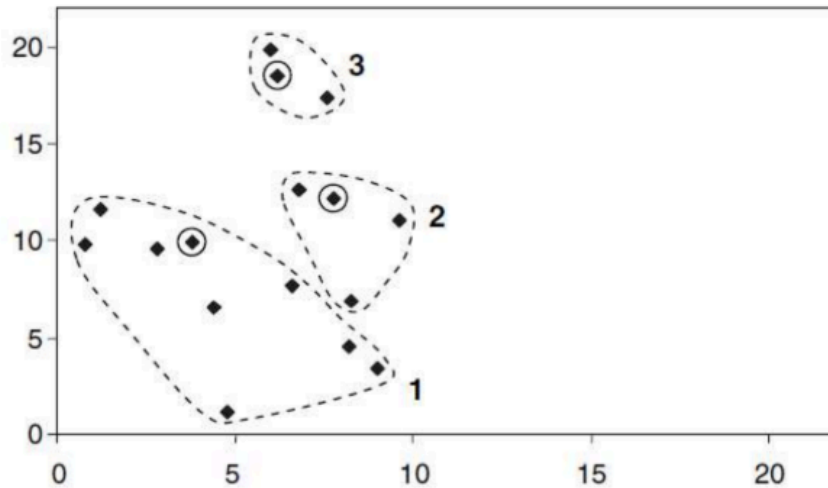
|            | Initial  |          |
|------------|----------|----------|
|            | <i>x</i> | <i>y</i> |
| Centroid 1 | 3.8      | 9.9      |
| Centroid 2 | 7.8      | 12.2     |
| Centroid 3 | 6.2      | 18.5     |

The distance of the first point (6.8, 12.6) from the first centroid (3.8, 9.9) is simply

$$\sqrt{(6.8 - 3.8)^2 + (12.6 - 9.9)^2} = 4.0 \text{ (to one decimal place)}$$

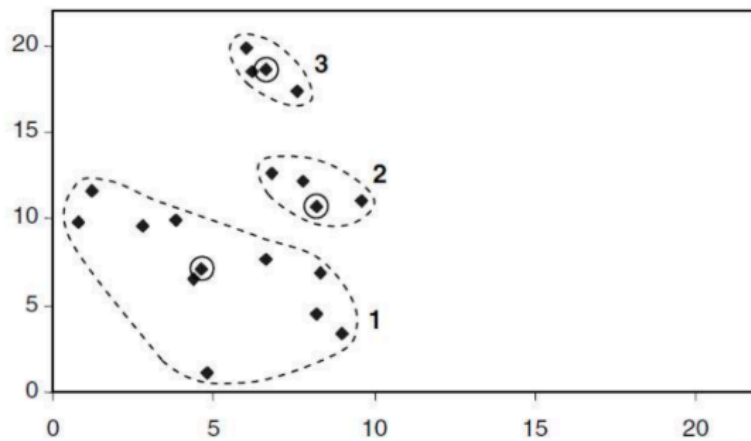
| <i>x</i> | <i>y</i> | <i>d1</i> | <i>d2</i> | <i>d3</i> | cluster |
|----------|----------|-----------|-----------|-----------|---------|
| 6.8      | 12.6     | 4.0       | 1.1       | 5.9       | 2       |
| 0.8      | 9.8      | 3.0       | 7.4       | 10.2      | 1       |
| 1.2      | 11.6     | 3.1       | 6.6       | 8.5       | 1       |
| 2.8      | 9.6      | 1.0       | 5.6       | 9.5       | 1       |
| 3.8      | 9.9      | 0.0       | 4.6       | 8.9       | 1       |
| 4.4      | 6.5      | 3.5       | 6.6       | 12.1      | 1       |
| 4.8      | 1.1      | 8.9       | 11.5      | 17.5      | 1       |
| 6.0      | 19.9     | 10.2      | 7.9       | 1.4       | 3       |
| 6.2      | 18.5     | 8.9       | 6.5       | 0.0       | 3       |
| 7.6      | 17.4     | 8.4       | 5.2       | 1.8       | 3       |
| 7.8      | 12.2     | 4.6       | 0.0       | 6.5       | 2       |
| 6.6      | 7.7      | 3.6       | 4.7       | 10.8      | 1       |
| 8.2      | 4.5      | 7.0       | 7.7       | 14.1      | 1       |
| 8.4      | 6.9      | 5.5       | 5.3       | 11.8      | 2       |
| 9.0      | 3.4      | 8.3       | 8.9       | 15.4      | 1       |
| 9.6      | 11.1     | 5.9       | 2.1       | 8.1       | 2       |

## The resulting clusters after 1<sup>st</sup> Iteration



## Centroids after 1<sup>st</sup> Iteration and Revised Cluster

|            | Initial |      | After first iteration |      |
|------------|---------|------|-----------------------|------|
|            | $x$     | $y$  | $x$                   | $y$  |
| Centroid 1 | 3.8     | 9.9  | 4.6                   | 7.1  |
| Centroid 2 | 7.8     | 12.2 | 8.2                   | 10.7 |
| Centroid 3 | 6.2     | 18.5 | 6.6                   | 18.6 |





## After 2<sup>nd</sup> Iteration and Third Set of Clusters

•These are the same clusters as before. Their centroids will be the same as those from which the clusters were generated.

•Hence the termination condition of the *k-means algorithm* 'repeat ... until the centroids no longer move' has been met and these are the final clusters produced by the algorithm for the initial choice of centroids made.

|            | Initial |      | After first iteration |      | After second iteration |      |
|------------|---------|------|-----------------------|------|------------------------|------|
|            | $x$     | $y$  | $x$                   | $y$  | $x$                    | $y$  |
| Centroid 1 | 3.8     | 9.9  | 4.6                   | 7.1  | 5.0                    | 7.1  |
| Centroid 2 | 7.8     | 12.2 | 8.2                   | 10.7 | 8.1                    | 12.0 |
| Centroid 3 | 6.2     | 18.5 | 6.6                   | 18.6 | 6.6                    | 18.6 |

